DAV Lab Programs

Lab Program 1: Introduction to Python for Data Analytics: NumPy and Pandas basics array operations, Data Frame creation, indexing, slicing.

PART A: NUMPY

```
# 1. Import NumPy
import numpy as np



# 2. Create a NumPy Array

arr = np.array([10, 20, 30, 40, 50])

print("NumPy Array:")
print(arr)



# 3. Array Indexing

print("\n--- Array Indexing ---")

print("First element:", arr[0])
print("Third element:", arr[2])
print("Last element:", arr[-1])



# 4. Array Slicing
```

```
print("\n--- Array Slicing ---")

print("First three elements:", arr[0:3])
print("Elements from index 2:", arr[2:])
print("Last three elements:", arr[-3:])


# 5. Basic Array Operations

print("\n--- Basic Array Operations ---")

print("Add 5:", arr + 5)
print("Subtract 5:", arr - 5)
print("Multiply by 2:", arr * 2)
print("Divide by 10:", arr / 10)


# 6. Mathematical Operations

print("\n--- Mathematical Operations ---")

print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
print("Maximum:", np.max(arr))
print("Minimum:", np.min(arr))


# 7. Reshaping an Array
```

```
print("\n--- Array Reshaping ---")

reshaped_arr = arr.reshape(5, 1)

print("Reshaped Array:")
print(reshaped_arr)


# PART B: PANDAS

# 8. Import Pandas
import pandas as pd


# 9. Create Sample Data

data = {
    "Name": ["Asha", "Ravi", "Meera", "Karan"],
    "Marks": [88, 72, 91, 65],
    "Branch": ["CSE", "ECE", "CSE", "ISE"]
}


# 10. Create a DataFrame

df = pd.DataFrame(data)

print("\n--- Pandas DataFrame ---")
print(df)
```

```
# 11. Column Indexing

print("\n--- Column Indexing ---")

print("Name Column:")
print(df["Name"])

print("\nMarks Column:")
print(df["Marks"])


# 12. Row Indexing

print("\n--- Row Indexing ---")

print("First Row:")
print(df.iloc[0])

print("\nThird Row:")
print(df.iloc[2])


# 13. Access a Specific Value

print("\n--- Specific Value ---")

print("Marks of first student:", df.iloc[0, 1])
```

```
# 14. DataFrame Row Slicing

print("\n--- Row Slicing ---")

print("First two rows:")
print(df.iloc[0:2])


# 15. Row and Column Slicing

print("\n--- Row and Column Slicing ---")

print("First three rows with Name and Marks:")
print(df.iloc[0:3, [0, 1]])


# 16. Filtering Data

print("\n--- Data Filtering ---")

result = df[df["Marks"] > 70]

print("Students with Marks greater than 70:")
print(result)
```

OUTPUT:

```
NumPy Array:
[10 20 30 40 50]

--- Array Indexing ---
First element: 10
Third element: 30
Last element: 50

--- Array Slicing ---
First three elements: [10 20 30]
Elements from index 2: [30 40 50]
Last three elements: [30 40 50]

--- Basic Array Operations ---
Add 5: [15 25 35 45 55]
Subtract 5: [ 5 15 25 35 45]
Multiply by 2: [ 20  40  60  80 100]
Divide by 10: [1. 2. 3. 4. 5.]

--- Mathematical Operations ---
Sum: 150
Mean: 30.0
Maximum: 50
Minimum: 10

--- Array Reshaping ---
Reshaped Array:
[[10]
 [20]
 [30]
 [40]
 [50]]

--- Pandas DataFrame ---
    Name  Marks Branch
0   Asha     88    CSE
1   Ravi     72    ECE
2  Meera     91    CSE
3  Karan     65    ISE
```

```
--- Column Indexing ---
Name Column:
0     Asha
1     Ravi
2    Meera
3    Karan
Name: Name, dtype: str

Marks Column:
0    88
1    72
2    91
3    65
Name: Marks, dtype: int64

--- Row Indexing ---
First Row:
Name      Asha
Marks       88
Branch     CSE
Name: 0, dtype: object

Third Row:
Name      Meera
Marks        91
Branch      CSE
Name: 2, dtype: object

--- Specific Value ---
Marks of first student: 88

--- Row Slicing ---
First two rows:
   Name  Marks Branch
0  Asha     88    CSE
1  Ravi     72    ECE

--- Row and Column Slicing ---
First three rows with Name and Marks:
    Name  Marks
0   Asha     88
1   Ravi     72
2  Meera     91

--- Data Filtering ---
Students with Marks greater than 70:
    Name  Marks Branch
0   Asha     88    CSE
1   Ravi     72    ECE
2  Meera     91    CSE
```

Lab Program 2: Exploratory Data Analysis: Loading datasets, computing descriptive statistics, identifying missing values and duplicates.

Dear students, for the laboratory program, you have to focus on Exploratory Data Analysis (EDA). First, understand how to load/create a dataset, inspect the data, calculate descriptive statistics, and identify missing values and duplicate records.

The flow is:

Dataset → Inspect → Understand → Statistics → Missing Values → Duplicates

So, this is your **actual EDA laboratory program**.

```
import pandas as pd


# 1. CREATE OUR OWN DATASET

df = pd.DataFrame({

    "Order Date": [
        "2024-01-10",
        "2024-01-15",
        "2024-02-05",
        "2024-02-12",
        "2024-03-01",
        "2024-03-15",
        "2024-04-10",
        "2024-04-20",
        "2024-05-05",
        "2024-01-15"
    ],
```

```
"Product Name": [
    "Laptop",
    "Mouse",
    "Printer",
    "Tablet",
    "Laptop",
    "Monitor",
    "Mouse",
    "Printer",
    "Smartwatch",
    "Mouse"
],

"Category": [
    "Electronics",
    "Accessories",
    "Office",
    "Electronics",
    "Electronics",
    "Accessories",
    "Accessories",
    "Office",
    "Electronics",
    "Accessories"
],

"Region": [
    "North",
    "South",
```

```
    "East",
    "West",
    "North",
    "South",
    "East",
    "West",
    "North",
    "South"
],

"Quantity": [
    2,
    5,
    3,
    4,
    1,
    2,
    6,
    2,
    3,
    5
],

"Sales": [
    50000,
    2500,
    9000,
    30000,
    55000,
```

```
        16000,
        3000,
        6000,
        45000,
        2500
    ],

    "Profit": [
        10000,
        500,
        1500,
        6000,
        11000,
        3000,
        None,
        1000,
        9000,
        500
    ]
})


# 2. ORIGINAL DATASET

print("ORIGINAL DATASET")
display(df)


# 3. FIRST 5 RECORDS
```

```
print("FIRST 5 RECORDS")
display(df.head())


# 4. DATASET SHAPE

print("DATASET SHAPE")
print(df.shape)


# 5. COLUMN NAMES

print("COLUMN NAMES")
print(df.columns)


# 6. DATA TYPES

print("DATA TYPES")
print(df.dtypes)


# 7. DESCRIPTIVE STATISTICS

print("DESCRIPTIVE STATISTICS")
display(df.describe())
```

```
# 8. MISSING VALUES

print("MISSING VALUES")
print(df.isnull().sum())


# 9. ROWS CONTAINING MISSING VALUES

print("ROWS CONTAINING MISSING VALUES")
display(df[df.isnull().any(axis=1)])


# 10. TOTAL DUPLICATE RECORDS

print("TOTAL DUPLICATE RECORDS")
print(df.duplicated().sum())


# 11. DUPLICATE RECORDS

print("DUPLICATE RECORDS")
display(df[df.duplicated()])
```

Lab Program: Loading a Real Dataset

```
import pandas as pd


# 1. Load the dataset

df = pd.read_csv("sales_data.csv")


# 2. Display the original dataset

print("ORIGINAL DATASET")
display(df)


# 3. Display the first 5 records

print("FIRST 5 RECORDS")
display(df.head())


# 4. Display the last 5 records

print("LAST 5 RECORDS")
display(df.tail())


# 5. Display the number of rows and columns
```

```
print("DATASET SHAPE")
print(df.shape)


# 6. Display column names

print("COLUMN NAMES")
print(df.columns)


# 7. Display data types

print("DATA TYPES")
print(df.dtypes)
```

DATA COMBINATION FOR BUSINESS UNDERSTANDING (This concept is not actual lab program but you have to understand the concept that's it)

Now that we have understood and explored the dataset, suppose the business gives us additional information stored in different tables. We need to combine that information to perform better analysis. This is called Data Combination.

It covers:

1. Concatenation
2. Merging
3. Joining
4. GroupBy
5. Pivot Table

```
import pandas as pd



# 1. CREATE SALES DATA


df = pd.DataFrame({


    "Order Date": [
        "2024-01-10",
        "2024-01-15",
        "2024-02-05",
        "2024-02-12",
        "2024-03-01",
        "2024-03-15",
        "2024-04-10",
        "2024-04-20",
        "2024-05-05",
        "2024-01-15"
```

```
],

"Product Name": [
    "Laptop",
    "Mouse",
    "Printer",
    "Tablet",
    "Laptop",
    "Monitor",
    "Mouse",
    "Printer",
    "Smartwatch",
    "Mouse"
],

"Category": [
    "Electronics",
    "Accessories",
    "Office",
    "Electronics",
    "Electronics",
    "Accessories",
    "Accessories",
    "Office",
    "Electronics",
    "Accessories"
],

"Region": [
```

```
    "North",
    "South",
    "East",
    "West",
    "North",
    "South",
    "East",
    "West",
    "North",
    "South"
],

"Quantity": [
    2,
    5,
    3,
    4,
    1,
    2,
    6,
    2,
    3,
    5
],

"Sales": [
    50000,
    2500,
    9000,
```

```
        30000,
        55000,
        16000,
        3000,
        6000,
        45000,
        2500
    ],

    "Profit": [
        10000,
        500,
        1500,
        6000,
        11000,
        3000,
        None,
        1000,
        9000,
        500
    ]
})


# 2. CONCATENATION

new_sales = pd.DataFrame({

    "Order Date": [
```

```
    "2024-06-01",
    "2024-06-05",
    "2024-06-10"
],

"Product Name": [
    "Laptop",
    "Mouse",
    "Printer"
],

"Category": [
    "Electronics",
    "Accessories",
    "Office"
],

"Region": [
    "North",
    "South",
    "East"
],

"Quantity": [
    2,
    4,
    3
],
```

```
    "Sales": [
        60000,
        2000,
        9000
    ],

    "Profit": [
        12000,
        400,
        1800
    ]
})


print("NEW SALES DATA")
display(new_sales)


combined_df = pd.concat(
    [df, new_sales],
    ignore_index=True
)


print("CONCATENATED DATA")
display(combined_df)


# 3. MERGING
```

```
product_info = pd.DataFrame({

    "Product Name": [
        "Laptop",
        "Mouse",
        "Printer",
        "Tablet",
        "Monitor"
    ],

    "Supplier": [
        "Dell",
        "Logitech",
        "HP",
        "Samsung",
        "LG"
    ],

    "Discount": [
        10,
        5,
        15,
        8,
        12
    ]
})
```

```
print("PRODUCT INFORMATION")
display(product_info)


merged_df = pd.merge(
    df,
    product_info,
    on="Product Name",
    how="left"
)


print("MERGED DATA")
display(merged_df)


# 4. JOINING

region_info = pd.DataFrame(

    {
        "Region_Manager": [
            "Manager A",
            "Manager B",
            "Manager C",
            "Manager D"
        ]
    },
```

```
    index=[
        "North",
        "South",
        "East",
        "West"
    ]
)


print("REGION INFORMATION")
display(region_info)


df_region = df.set_index("Region")


joined_df = df_region.join(region_info)


print("JOINED DATA")
display(joined_df)


# 5. GROUPBY

region_sales = df.groupby(
    "Region"
)["Sales"].sum()
```

```
print("TOTAL SALES BY REGION")
display(region_sales)



# 6. PIVOT TABLE

pivot_table = pd.pivot_table(

    df,
    values="Sales",
    index="Category",
    columns="Region",
    aggfunc="sum",
    fill_value=0
)


print("PIVOT TABLE")
display(pivot_table)
```